

ASTRAVUL: AST-Guided Retrieval-Augmented Generation for Automated Software Vulnerability Detection and Triage: A Literature Review

Abdullah Al Noman Kazi Neyamul Hasan Rakibul Hassan
Mahathir Mohammad Md. Habibulla Misba

Department of Computer Science and Engineering
United International University (UIU)

September 22, 2026

Abstract

Abstract—Automated vulnerability detection is vital for modern software security. Traditional Static Application Security Testing (SAST) suffers from high False Discovery Rates ($FDR \geq 40\%$), while naive querying of Large Language Models (LLMs) on raw source code introduces intellectual property risks, severe context saturation, and hallucinated reports. Recent research converges on hybrid neuro-symbolic paradigms: deterministic program slicing via Abstract Syntax Trees (AST) combined with knowledge-level Retrieval-Augmented Generation (RAG) and local, privacy-preserving LLM verification. This paper synthesizes 15 foundational and state-of-the-art Q1 papers (including USENIX Security, NDSS, ICSE, ACM TOSEM, IEEE TDSC, NeurIPS, and ACL) into a unified taxonomy, identifying critical research gaps and establishing the foundation for **ASTRAVUL**, a token-efficient, high-precision triage pipeline.

Keywords—Vulnerability Detection, Abstract Syntax Tree, Program Slicing, Retrieval-Augmented Generation, Local LLM, ChromaDB, Neuro-Symbolic Security, AstraVul.

1 Introduction

Modern software vulnerability detection has become indispensable in the Secure Software Development Life Cycle (SSDLC). However, current solutions exhibit critical limitations:

1. **Heuristic Static Analysis (SAST):** Pattern-matching tools (e.g., Flawfinder, Cppcheck) generate excessive false positives ($FDR \geq 40\%$) due to a lack of semantic context and dataflow awareness.
2. **Graph-Based Deep Learning:** Graph Neural Networks (e.g., Devign [3], SySeVR [2]) capture structural relationships but operate as uninterpretable black boxes and struggle with out-of-distribution code.
3. **Naive Large Language Models (LLMs):** While LLMs offer powerful reasoning, recent empirical audits (e.g., PrimeVul [14], Chasing Shadows [15]) show they suffer from “patch-blindness”, attention degradation on large files, and enterprise data privacy risks.

To address these challenges, we review 15 seminal Q1 papers and establish the architectural motivation for **ASTRAVUL**, an AST-guided, RAG-grounded neuro-symbolic triage engine.

2 Thematic Synthesis of Reviewed Literature

We categorize the reviewed literature into three complementary technical pillars:

1. Program Slicing and Graph Representations: Early deep learning on source code was hampered by excessive syntactical noise in whole-function representations. Li et al. [1] introduced *code gadgets* in VulDeePecker, demonstrating that slicing backward dataflow around library/API call sinks significantly outperforms full-function classification. SySeVR [2] generalized this by defining Syntax-, Semantics-, and Vulnerability-related Slices (SeVCs) across Control Flow Graphs (CFG) and Data Flow Graphs (DFG), detecting 15 zero-day vulnerabilities in open-source software. Devign [3] established that ASTs alone are insufficient; composite Code Property Graphs (CPGs) combining AST, CFG, and DFG embedded via Gated Graph Neural Networks (GGNN) are necessary for complex memory flaws. DeepDFA [10] enriched graph embeddings with compiler-level dataflow bit-vectors, making models resilient to semantic-preserving renaming. Recently, Lekssays et al. [13] (LLMxCPG) showed that deterministic CPG slicing reduces LLM input tokens by 67.8%–90.9% while boosting F1-scores by 15%–40%.

2. Benchmark Realities and Empirical LLM Fragility: Initial high accuracy claims in vulnerability detection were severely undermined by benchmark artifacts. ReVeal [4] revealed that models trained on synthetic benchmarks (e.g., NIST Juliet) overfit to artificial syntax, experiencing catastrophic precision drops (collapsing to 10%–14%) in production repositories. DiverseVul [8] audited historical corpora (e.g., Big-Vul) and discovered up to 48% label noise, establishing a cleaned benchmark of 18,945 real-world C/C++ vulnerabilities. CVEfixes [7] automated the harvesting of ground-truth fix commits from NVD, proving that contrastive pre-patch/post-patch pairs are critical. In the LLM domain, PrimeVul [14] showed that open-source 7B models achieving 68.3% F1 on legacy datasets collapsed to 3.1% F1 on debiased real-world data. In Chasing Shadows, Arp et al. [15] surveyed 72 security papers, formalizing 9 systemic pitfalls and demonstrating that models must be evaluated using the False Discovery Rate ($FDR = FP / (TP + FP)$).

3. Pre-trained Embeddings, RAG, and Neuro-Symbolic Verification: Pre-trained transformers advanced code comprehension: UniXcoder [6] unified comments, flattened AST prefixes, and code tokens in a 768-dimensional multimodal

space. LineVul [5] utilized CodeBERT self-attention weights to localize flaws to exact lines, reducing developer effort by $> 80\%$. PDBERT [11] pre-trained on Control and Data Dependency Prediction, achieving $23\times$ higher throughput than heavy static analyzers. In neuro-symbolic triage, GPTScan [9] combined static candidate extraction with LLM constraint validation ($> 90\%$ precision). Addressing LLM “patch-blindness”, Vul-RAG [12] injected external CWE rules, CVE descriptions, and patch diffs into the prompt, boosting accuracy by $16\%–24\%$.

Table 1 provides a structured comparative taxonomy synthesizing these 15 seminal works across analysis paradigm, program representation, retrieval mechanism, privacy model, and addressed limitation.

Research Gaps and ASTRAVUL Architecture: This synthesis exposes three central gaps: (1) *Context Saturation*: Raw file prompts dilute attention; ASTRAVUL’s deterministic Tree-sitter AST slicing prunes $\geq 70\%$ of unneeded tokens into minimal ≤ 150 -token slices. (2) *Patch-Blindness*: General LLMs hallucinate false alarms on patched code; grounding slices via ChromaDB indexed with NIST CWE specifications and DiverseVul/CVEfixes diffs provides explicit boundary invariants. (3) *Privacy and Hallucination*: Cloud LLM reliance risks code leaks; ASTRAVUL’s local 4-bit quantized models (Qwen2.5-Coder-7B) coupled with Pydantic JSON schema decoding guarantee verifiable, zero-leakage triage reports.

3 Conclusion

This literature review establishes the foundation for AST-guided RAG in software vulnerability detection. Uniting deterministic AST program slicing, semantic knowledge retrieval, and local constrained LLM triage directly resolves the core trade-offs between precision, speed, explainability, and code privacy.

References

- [1] Z. Li et al., “VulDeePecker: A deep learning-based system for vulnerability detection,” in *Proc. NDSS*, 2018.
- [2] Z. Li et al., “SySeVR: A framework for using deep learning to detect software vulnerabilities,” *IEEE TDSC*, 19(4):2244–2258, 2021.
- [3] Y. Zhou et al., “Devign: Effective vulnerability identification via graph neural networks,” in *Proc. NeurIPS*, 2019.
- [4] S. Chakraborty et al., “Deep learning for vulnerability detection: Are we there yet?,” in *Proc. ICSE*, pp. 883–895, 2021.
- [5] M. Fu and C. Tantithamthavorn, “LineVul: A transformer-based line-level vulnerability prediction,” in *Proc. MSR*, pp. 608–620, 2022.
- [6] D. Guo et al., “UniXcoder: Unified cross-modal pre-training for code representation,” in *Proc. ACL*, pp. 7212–7225, 2022.
- [7] G. Bhandari et al., “CVEfixes: Automated collection of vulnerabilities and their fixes,” in *Proc. MSR*, pp. 241–245, 2021.
- [8] Y. Chen et al., “DiverseVul: A new vulnerable source code dataset for deep learning,” in *Proc. RAID*, pp. 654–668, 2023.
- [9] Y. Sun et al., “GPTScan: Detecting logic vulnerabilities by combining GPT with program analysis,” in *Proc. ICSE*, 2024.
- [10] B. Steenhoeck et al., “Dataflow analysis-inspired deep learning for vulnerability detection,” in *Proc. ICSE*, 2024.
- [11] Z. Liu et al., “Pre-training by predicting program dependencies for vulnerability analysis,” in *Proc. ICSE*, 2024.
- [12] X. Du et al., “Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG,” *ACM TOSEM*, 2024.
- [13] A. Lekssays et al., “LLMxCPG: Context-aware vulnerability detection through CPG-guided LLMs,” in *Proc. USENIX Security*, 2025.
- [14] Y. Zhang et al., “Vulnerability detection with code language models: How far are we?,” in *Proc. ICSE*, 2025.
- [15] D. Arp et al., “Chasing shadows: Pitfalls in LLM security research,” in *Proc. NDSS*, 2026.

Table 1: Systematic Comparative Taxonomy of Reviewed Literature

Paper / System	Approach Paradigm	Slicing/AST	RAG	Privacy / Local	Key Addressed Limitation
VulDeePecker [1]	Deep Learning (BLSTM)	API Sinks	No	Offline Model	Coarse file-level analysis
SySeVR [2]	Deep Learning (BBiRNN)	AST+CFG+DFG	No	Offline Model	Missing control-flow context
Devign [3]	Graph Neural Net (GGNN)	Composite CPG	No	Offline Model	Syntax-only representation
ReVeal [4]	Graph DL / Empirical	Graph Slices	No	Offline Model	Exposed synthetic dataset bias
UniXcoder [6]	Cross-Modal Pre-training	AST Flattening	No	Local Weights	Semantic code embedding gap
LineVul [5]	Transformer (CodeBERT)	Token Sequence	No	Local Weights	Coarse function-level alerts
CVEfixes [7]	Curated Security Corpus	Function/Diff	No	N/A	Ground-truth CVE-to-fix mapping
DiverseVul [8]	Benchmark Evaluation	Function Level	No	N/A	Mitigated label noise across 150 CWEs
GPTScan [9]	Neuro-Symbolic Hybrid	Static Analysis	No	Cloud API (GPT-3.5)	> 90% precision in logic flaw verification
DeepDFA [10]	Neuro-Symbolic / GNN	Dataflow Vectors	No	Local Weights	Injected symbolic dataflow into embeddings
PDBERT [11]	Pre-trained Transformer	Program Dep.	No	Local Weights	23× faster dependency-aware throughput
Vul-RAG [12]	Retrieval-Augmented LLM	No	Yes	Cloud API (GPT-4)	Overcame LLM patch-blindness (+16–24%)
LLMxCPG [13]	Neuro-Symbolic Hybrid	CPG Slicing	No	Local Fine-Tuning	68–91% code reduction; +15–40% F1 score
PrimeVul [14]	Empirical Benchmarking	Function Level	No	Local & Cloud LLMs	Proved raw LLMs fail on debiased datasets
Chasing Shadows [15]	Systematic Review	N/A	No	N/A	Formalized 9 pitfalls; highlighted FDR metric
ASTRAVUL (Ours)	AST-Guided RAG+LLM	AST+DFG ($\leq 150t$)	Yes	Local 4-bit Quant.	Resolves FDR, hallucination, & privacy